

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Estudios Superiores Acatlán

Licenciatura en Ciencia de Datos

Proyecto Final: HitSignal

Detección de *hidden gems* y búsqueda de similitud acústica con
PySpark

Materia: Datos Masivos

Profesora: Alondra Vanianinetl Berzunza Rodríguez

Alumno: Luis Alfredo Ramírez Maza

22 de mayo de 2026

Índice

1. Resumen	2
2. Introducción	2
3. Planteamiento del problema	3
4. Objetivos	3
4.1. Objetivo general	3
4.2. Objetivos específicos	3
5. Hipótesis	4
6. Marco teórico	4
6.1. Datos masivos y procesamiento distribuido	4
6.2. Popularidad musical como variable observable	4
6.3. Perfil acústico de una canción	5
6.4. Hidden gem	5
6.5. Búsqueda de similitud	6
7. Materiales y método	6
7.1. Datos utilizados	6
7.2. Limpieza de canciones	6
7.3. Variables derivadas	7
7.4. Análisis exploratorio	7
7.5. Construcción de la puntuación acústica	8
7.6. Limpieza y uso de <code>artists.csv</code>	9
7.7. Join entre canciones y artistas	10
8. Desarrollo experimental	10
8.1. Definición final de Hidden Gems V3	10
8.2. Deduplicación	11
8.3. Búsqueda exacta de similitud	11
8.4. Búsqueda aproximada con LSH	12

9. Resultados	12
9.1. Resultados de limpieza y EDA	12
9.2. Resultados del enriquecimiento con artistas	12
9.3. Resultados de Hidden Gems V3	12
9.4. Resultados de búsqueda exacta	13
9.5. Resultados de LSH	13
10.Discusión	14
11.Limitaciones	15
12.Conclusiones	15
13.Bibliografía y recursos	16
A. Anexo: código esencial en PySpark	17
A.1. Limpieza de tracks y variables base	17
A.2. Puntuación acústica	17
A.3. Join entre tracks y artistas	18
A.4. Definición final de Hidden Gems V3	19
A.5. Recencia, score final y deduplicación	20
A.6. Búsqueda exacta de similitud	20
A.7. Búsqueda aproximada con LSH	21

1. Resumen

El proyecto **HitSignal** desarrolla un flujo de procesamiento de datos musicales con PySpark para identificar canciones poco populares con alto potencial acústico. La propuesta parte de un conjunto de datos de Spotify con canciones y artistas, realiza limpieza, análisis exploratorio, enriquecimiento mediante joins con información de artistas y construye una definición final de *hidden gems*.

La idea central es que una canción no debe considerarse *hidden gem* solamente por tener baja popularidad en el campo `popularity`. En catálogos musicales, una versión, reedición, remix o registro con bajo conteo local puede pertenecer a un artista conocido. Por ello, HitSignal incorpora información de `artists.csv`, especialmente popularidad y seguidores de los artistas, para filtrar canciones asociadas con artistas masivos.

La salida principal es una lista de 11,613 canciones finales que cumplen simultáneamente condiciones de baja popularidad del track, lanzamiento relativamente reciente, perfil acústico alto y artista no masivo. Posteriormente, estas canciones se comparan contra hits modernos mediante búsqueda de similitud exacta y una demostración con LSH sobre una muestra estratificada representativa. El resultado permite asociar cada *hidden gem* con un hit acústicamente cercano, convirtiendo el análisis en una herramienta interpretable para descubrimiento musical.

Palabras clave: PySpark, datos masivos, Spotify, hidden gems, joins, similitud acústica, LSH.

2. Introducción

La industria musical digital genera grandes volúmenes de datos. Cada canción puede estar acompañada de metadatos, métricas de popularidad, características acústicas y relaciones con artistas. Esta disponibilidad de información permite construir sistemas analíticos que ayuden a estudiar patrones de popularidad y a descubrir canciones que aún no tienen alta visibilidad.

En este contexto surge **HitSignal**, un prototipo académico de datos masivos cuyo propósito es detectar canciones con potencial musical a partir de variables cuantitativas. El proyecto no pretende predecir de forma absoluta qué canción será exitosa, sino construir una metodología reproducible para encontrar canciones poco populares que comparten características acústicas con canciones exitosas y que, además, no pertenecen a artistas masivos.

El trabajo se desarrolló en PySpark por la naturaleza tabular y volumétrica de los datos. Primero se prepararon y limpiaron los datos de canciones. Después se hizo análisis exploratorio para identificar variables acústicas relevantes. Posteriormente se incorporó información de artistas mediante joins, lo cual fue necesario para corregir falsos positivos

en la definición de *hidden gems*. Finalmente se realizó una búsqueda de similitud para conectar cada canción candidata con hits modernos acústicamente cercanos.

3. Planteamiento del problema

Una definición ingenua de *hidden gem* podría considerar solamente canciones con baja popularidad. Sin embargo, este criterio es insuficiente. En el dataset de Spotify, la variable `popularity` está asociada al `track_id`, no necesariamente al impacto histórico de la canción o del artista. Por ello, pueden aparecer registros con baja popularidad local que en realidad pertenecen a artistas ampliamente conocidos.

El problema principal es distinguir entre:

- canciones realmente poco conocidas con potencial acústico, y
- canciones de baja popularidad local que pertenecen a artistas grandes, reediciones, versiones, remixes o registros secundarios.

Para resolver esta ambigüedad fue necesario utilizar `artists.csv`. La información de artistas permitió incorporar variables como popularidad del artista y número de seguidores. Con esto, una canción solo puede considerarse candidata si tanto el track como sus artistas cumplen condiciones de baja exposición.

4. Objetivos

4.1. Objetivo general

Diseñar un pipeline en PySpark para detectar canciones poco populares con alto potencial acústico, filtrar artistas masivos mediante enriquecimiento con `artists.csv` y asociar las canciones finales con hits modernos acústicamente cercanos.

4.2. Objetivos específicos

- Limpiar y preparar el dataset de canciones conservando valores válidos de variables acústicas.
- Analizar la distribución de popularidad y las diferencias acústicas entre canciones hit y no hit.
- Construir una puntuación acústica experimental basada en variables asociadas con canciones populares.

- Limpiar `artists.csv` y construir una tabla de referencia para joins.
- Enriquecer cada track con información agregada de sus artistas.
- Definir la versión final de *hidden gems* incorporando filtros de track, artista, recencia y deduplicación.
- Implementar búsqueda de similitud para encontrar hits cercanos a cada *hidden gem*.

5. Hipótesis

Si se combinan variables acústicas asociadas con canciones populares y se filtran canciones cuyos artistas no son masivos, entonces es posible construir una lista de *hidden gems* más estricta que aquella basada solamente en baja popularidad del track.

Adicionalmente, si cada canción se representa como un vector de características acústicas normalizadas, entonces es posible encontrar hits modernos cercanos a las *hidden gems* mediante distancia euclidiana ponderada y búsqueda aproximada con LSH.

6. Marco teórico

6.1. Datos masivos y procesamiento distribuido

Los datos masivos requieren herramientas capaces de procesar grandes volúmenes de información de forma eficiente. PySpark permite expresar transformaciones distribuidas sobre DataFrames, incluyendo filtrados, agregaciones, joins y escritura en formatos columnares como Parquet. En este proyecto, Spark se utilizó para construir un flujo ETL reproducible desde archivos CSV hasta resultados finales.

6.2. Popularidad musical como variable observable

La variable `popularity` de Spotify se encuentra en una escala de 0 a 100. En este proyecto se interpretó como una medida observable de exposición del track dentro del dataset. No se asume como calidad musical ni como *éxito* histórico absoluto, sino como una señal cuantitativa para clasificar canciones.

El percentil 99 de popularidad fue 70, por lo que se definió una canción hit como aquella con:

$$\text{is_hit} = 1 \iff \text{popularity} \geq 70. \quad (1)$$

Esta definición es estricta: no toda canción popular es considerada hit, solo aquellas en la parte superior de la distribución.

6.3. Perfil acústico de una canción

Spotify proporciona variables acústicas como `danceability`, `energy`, `loudness`, `acousticness` e `instrumentalness`. El EDA mostró que estas variables diferencian de forma razonable canciones hit y no hit. Por ello, se usaron para construir una puntuación acústica experimental.

Para cada variable continua se aplicó normalización. En el caso de variables con relación negativa con popularidad, se utilizó el complemento:

$$c_j = 1 - x_j. \quad (2)$$

Para variables con relación positiva, se usó directamente:

$$c_j = x_j. \quad (3)$$

La puntuación acústica final puede verse como una combinación ponderada:

$$S = 100 \cdot \sum_{j=1}^m w_j c_j, \quad (4)$$

donde w_j representa el peso asignado a la variable j y c_j es su componente normalizado o invertido.

6.4. Hidden gem

En este proyecto, una *hidden gem* se define como una canción que combina cuatro propiedades:

1. baja popularidad del track,
2. perfil acústico alto,
3. lanzamiento relativamente reciente,
4. artistas no masivos.

El cuarto criterio es esencial. Sin información de artistas, el sistema podría clasificar como *hidden gem* una canción poco reproducida localmente pero asociada a un artista globalmente conocido.

6.5. Búsqueda de similitud

La búsqueda de similitud representa cada canción como un vector numérico. Si $\mathbf{x}$ representa una *hidden gem* y $\mathbf{y}$ representa un hit, la distancia euclidiana ponderada se define como:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{j=1}^m w_j (x_j - y_j)^2}. \quad (5)$$

Una distancia menor indica mayor similitud acústica según las variables seleccionadas. Para escalar esta búsqueda se usó LSH, una técnica que agrupa vectores similares en buckets cercanos para reducir comparaciones innecesarias.

7. Materiales y método

7.1. Datos utilizados

Se trabajó con dos archivos principales:

- `tracks.csv`: canciones, metadatos, popularidad y características acústicas.
- `artists.csv`: identificador, nombre, popularidad, seguidores y géneros de artistas.

El conjunto de canciones contenía originalmente 586,672 tracks. El conjunto de artistas contenía 1,162,095 registros. Los archivos procesados se almacenaron en Parquet para reutilizar resultados intermedios.

7.2. Limpieza de canciones

La limpieza fue conservadora. Se eliminaron solamente registros con errores claros:

- canciones sin nombre,
- `tempo` ≤ 0 ,
- `time_signature` ≤ 0 .

No se eliminaron valores cero en variables acústicas, porque pueden ser válidos. Tampoco se eliminaron valores negativos de `loudness`, ya que esta variable se mide en decibeles y normalmente puede tomar valores negativos.

Elemento	Cantidad
Filas originales en tracks	586,672
Filas eliminadas	408
Filas limpias	586,264
Porcentaje eliminado	0.0695 %
Duplicados por id	0

Tabla 1: Resumen de limpieza del dataset de canciones.

7.3. Variables derivadas

A partir de `tracks_clean` se construyó una tabla base con variables derivadas:

- `release_year`: año extraído de `release_date`.
- `release_decade`: década de lanzamiento.
- `duration_min`: duración en minutos.
- `is_hit`: indicador de hit, definido por `popularity >= 70`.
- `popularity_class`: baja, media o alta popularidad.

El umbral `popularity >= 70` se justificó porque coincide con el percentil 99 de la distribución de popularidad.

Percentil	Popularity
p50	27
p75	41
p90	52
p95	59
p99	70

Tabla 2: Percentiles de popularidad usados para definir el umbral de hit.

7.4. Análisis exploratorio

El EDA se enfocó en tres preguntas:

1. ¿cómo se distribuye la popularidad?

2. ¿qué variables acústicas distinguen hits de no hits?
3. ¿qué variables pueden servir para construir un perfil acústico de potencial?

La distribución de popularidad se concentró en valores bajos: 73.13 % de canciones quedaron en clase baja, 25.62 % en clase media y solo 1.25 % en clase alta.

Clase	Número de canciones	Porcentaje
Baja	428,759	73.13 %
Media	150,191	25.62 %
Alta	7,314	1.25 %

Tabla 3: Distribución de clases de popularidad.

Al comparar hits y no hits se observaron patrones acústicos claros: los hits tuvieron menor *acousticness*, menor *instrumentalness*, mayor *energy*, mayor *loudness* y mayor *danceability*.

Variable	No hit	Hit	Diferencia
<i>acousticness</i>	0.4523	0.2456	-0.2067
<i>danceability</i>	0.5628	0.6509	0.0881
<i>energy</i>	0.5410	0.6437	0.1027
<i>loudness</i>	-10.2433	-6.6776	3.5657
<i>instrumentalness</i>	0.1144	0.0229	-0.0915

Tabla 4: Comparación de características promedio entre canciones hit y no hit.

7.5. Construcción de la puntuación acústica

Con base en el EDA se construyó una puntuación acústica para medir qué tan cercano es un track al perfil promedio observado en canciones populares. Las variables seleccionadas fueron:

- *danceability*,
- *energy*,
- *loudness*,
- *acousticness*,

- `instrumentalness`.

Las variables con asociación positiva con popularidad aportan directamente al score. Las variables con asociación negativa se invierten. Por ejemplo, como `acousticness` e `instrumentalness` tienden a ser menores en hits, sus componentes se calcularon como complemento.

Variable	Correlación	Peso	Dirección
<code>acousticness</code>	-0.3710	0.2599	negativa
<code>loudness</code>	0.3285	0.2302	positiva
<code>energy</code>	0.3029	0.2122	positiva
<code>instrumentalness</code>	-0.2372	0.1662	negativa
<code>danceability</code>	0.1878	0.1316	positiva

Tabla 5: Variables usadas para la puntuación acústica.

La puntuación no se interpreta como predicción comercial de *hit*; funciona como una señal acústica para ordenar candidatos.

7.6. Limpieza y uso de `artists.csv`

La tabla de artistas fue necesaria porque la popularidad del track no basta para identificar canciones realmente poco expuestas. Se verificó que `id` fuera una llave confiable: no hubo IDs nulos ni duplicados. La variable `followers` tuvo solamente 11 nulos.

Métrica	Valor
Artistas totales	1,162,095
IDs únicos	1,162,095
IDs duplicados	0
Followers nulos	11
Popularity fuera de rango	0
Followers negativos	0

Tabla 6: Diagnóstico de limpieza de artistas.

Se construyó una tabla compacta `artists_ref` con las columnas esenciales para el join: `artist_id`, `artist_name`, `artist_popularity`, `artist_followers`, clases de popularidad y clases de tamaño.

7.7. Join entre canciones y artistas

El campo `id_artists` de `tracks` fue parseado como arreglo. Luego se aplicó `explode` para crear una tabla track-artista y se hizo join con `artists_ref`. El resultado tuvo un match alto: 96.56 % por fila track-artista y 98.22 % por track.

Métrica	Valor
Filas track-artista	756,555
Artistas únicos en tracks	98,400
Match rate por fila	96.56 %
Tracks con al menos un match	575,848
Tracks sin match	10,416
Match rate por track	98.22 %

Tabla 7: Calidad del join entre tracks y artistas.

Para cada canción se agregaron variables a nivel track, como `max_artist_popularity` y `max_artist_followers`. Se utilizó el máximo porque si una canción tiene varios artistas y al menos uno es masivo, entonces no debe considerarse una *hidden gem* estricta.

8. Desarrollo experimental

8.1. Definición final de Hidden Gems V3

La versión final de *hidden gems* se construyó integrando evidencia del track, del artista y del tiempo. Una canción se consideró candidata si cumplía simultáneamente:

$$\text{popularity} < 40, \quad (6)$$

$$\text{release_year} \geq 2010, \quad (7)$$

$$\text{hit_score_weighted} \geq 80, \quad (8)$$

$$\text{max_artist_popularity} < 60, \quad (9)$$

$$\text{max_artist_followers} < 500000, \quad (10)$$

$$\text{has_artist_match} = \text{true}. \quad (11)$$

Cada criterio tiene una función específica. El filtro de popularidad garantiza que el track sea poco visible. El filtro de año evita que el ranking esté dominado por canciones muy antiguas. El score acústico identifica canciones con perfil similar al de hits. Los filtros de artista evitan que canciones de artistas masivos pasen como oportunidades ocultas.

Para incorporar recencia se calculó la edad de la canción:

$$\text{song_age} = \text{max_year} - \text{release_year}. \quad (12)$$

Luego se aplicó una función de decaimiento exponencial con vida media de 10 años:

$$\text{recency_score} = e^{-\lambda \cdot \text{song_age}} \times 100, \quad \lambda = \frac{\ln 2}{10}. \quad (13)$$

Finalmente, se definió el ranking de oportunidad:

$$\text{hidden_gem_score} = 0,80 \cdot \text{hit_score_weighted} + 0,20 \cdot \text{recency_score}. \quad (14)$$

El peso 80/20 conserva como criterio principal el perfil acústico y usa la recencia solamente como ajuste. Esta decisión evita que el ranking se convierta en una lista de canciones recientes sin suficiente similitud acústica con hits.

8.2. Deduplicación

Una vez filtradas las candidatas, se normalizó el nombre de la canción y se construyó una llave de artistas. Esto permitió eliminar versiones equivalentes o duplicadas del mismo track.

La llave de deduplicación fue:

$$\text{dedup_key} = (\text{track_name_norm}, \text{artist_key}). \quad (15)$$

Dentro de cada grupo se conservó la canción con mejor score, priorizando el ranking final.

8.3. Búsqueda exacta de similitud

Con la lista final de *hidden gems*, se construyó una búsqueda exacta sobre muestra. Se tomaron 500 hits modernos y 500 *hidden gems*. Esto equivale a:

$$500 \times 500 = 250000 \quad (16)$$

comparaciones potenciales. Para cada par se calculó distancia euclidiana ponderada y, por cada *hidden gem*, se conservó el hit con menor distancia.

Esta etapa funciona como validación conceptual: demuestra que la representación vec-

torial puede conectar canciones poco populares con hits específicos.

8.4. Búsqueda aproximada con LSH

Para extender la búsqueda a un escenario más grande se implementó `BucketedRandomProjectionLSH`. A diferencia de la búsqueda exacta, LSH no evalúa explícitamente todos los pares: genera candidatos probables a partir de buckets de vectores similares.

La muestra LSH fue estratificada, no seleccionada por ranking. Para hits se estratificó por año y rango de popularidad. Para *hidden gems* se estratificó por año, popularidad y `hidden_gem_score`. Esto evitó que la demostración dependiera solamente de los ejemplos más altos del ranking.

La muestra usada para LSH fue:

$$801 \times 1204 = 964404 \tag{17}$$

pares potenciales, casi cuatro veces más que la búsqueda exacta de muestra.

9. Resultados

9.1. Resultados de limpieza y EDA

La limpieza produjo 586,264 canciones finales. El EDA confirmó que la popularidad está concentrada en valores bajos, por lo que detectar canciones de oportunidad requiere distinguir entre baja popularidad genuina y registros secundarios de artistas conocidos.

Las variables acústicas seleccionadas mostraron diferencias consistentes entre hits y no hits. En particular, las canciones hit fueron, en promedio, más energéticas, más bailables, más ruidosas y menos acústicas o instrumentales.

9.2. Resultados del enriquecimiento con artistas

El join con `artists.csv` fue una parte central del proyecto. La calidad del match permitió incorporar variables de artista a la mayoría de los tracks. Esto resolvió el problema de falsos positivos: canciones de baja popularidad local pero pertenecientes a artistas populares podían ser descartadas usando `max_artist_popularity` y `max_artist_followers`.

9.3. Resultados de Hidden Gems V3

La construcción final redujo progresivamente el universo de candidatos:

Etapa	Número de tracks
Candidatos con filtros solo a nivel track	20,226
Candidatos después de filtros de artista	11,613
Hidden gems finales depuradas	11,613

Tabla 8: Construcción de la lista final depurada de Hidden Gems V3.

La salida final es una tabla de canciones poco populares, recientes, de artistas no masivos y con alto perfil acústico. El conteo reportado corresponde al conjunto final depurado usado en la presentación y en el dashboard. Esta lista se usó como entrada para la búsqueda de similitud.

9.4. Resultados de búsqueda exacta

La búsqueda exacta sobre 250,000 pares potenciales produjo una tabla de 500 pares finales, uno por cada *hidden gem* de la muestra. La mediana de distancia fue 0.0215.

Métrica	Valor
Pares finales	500
Distancia mínima	0.0037
Percentil 25	0.0149
Mediana	0.0215
Percentil 75	0.0311
Distancia máxima	0.1368

Tabla 9: Distribución de distancias en la búsqueda exacta de muestra.

La correlación entre distancia y score acústico de la *hidden gem* fue baja. Esto es razonable: el score mide alineación con un perfil general de hits, mientras que la distancia mide cercanía con un hit específico.

9.5. Resultados de LSH

La muestra LSH conservó promedios muy cercanos a los conjuntos completos.

Dataset	n	Año prom.	Pop. prom.	Score acústico prom.
Hits completos	5,564	2017.97	74.84	76.18
Muestra hits	801	2017.98	74.75	75.57

Tabla 10: Validación de representatividad de la muestra LSH de hits.

Dataset	n	Año	Pop.	Score acústico	HGS
Hidden gems completas	11,613	2014.29	26.71	84.98	80.83
Muestra hidden gems	1,204	2014.30	26.81	84.86	80.74

Tabla 11: Validación de representatividad de la muestra LSH de hidden gems. HGS corresponde a `hidden_gem_score`.

LSH encontró un hit cercano para las 1,204 *hidden gems* de la muestra.

Métrica	Valor
Hits en muestra LSH	801
Hidden gems en muestra LSH	1,204
Pares potenciales	964,404
Hidden gems con match LSH	1,204
Cobertura	100 %
Distancia mínima	0.0038
Distancia máxima	0.1180

Tabla 12: Resultados de la demostración con LSH.

10. Discusión

El resultado más importante del proyecto no fue solo generar una lista de canciones con baja popularidad, sino corregir una debilidad metodológica: la baja popularidad a nivel track no implica que una canción sea desconocida. El uso de `artists.csv` permitió enriquecer cada canción con popularidad y seguidores de sus artistas, produciendo una definición más estricta de *hidden gem*.

El uso del máximo de popularidad y seguidores por track es una decisión conservadora. Si una canción tiene varios artistas, basta con que uno sea masivo para que la canción deje de ser considerada oportunidad oculta. Esto reduce falsos positivos y fortalece la interpretabilidad del resultado.

La búsqueda de similitud complementa la lista final. Mientras el score acústico dice que una canción se parece al perfil general de canciones populares, la distancia permite responder una pregunta más concreta: *¿a qué hit se parece?* Esta salida es más útil para una narrativa de descubrimiento musical.

La demostración con LSH no reemplaza la búsqueda exacta; la complementa. La búsqueda exacta valida el concepto en una muestra controlada. LSH muestra cómo escalar la idea a un espacio de comparación mayor sin plantear directamente un cross join exhaustivo.

11. Limitaciones

- La variable `popularity` es temporal y depende del momento de captura del dataset.
- La puntuación acústica no predice *hits* reales; solo mide similitud con un perfil observado.
- La similitud calculada no incorpora letra, idioma, contexto cultural, marketing, género explícito ni recepción del público.
- La variable `genres` de artistas está incompleta, por lo que no se usó como filtro principal.
- LSH devuelve vecinos aproximados y puede omitir algunos vecinos exactos.
- La demostración LSH se ejecutó sobre una muestra estratificada, no sobre todos los pares posibles; sin embargo, la muestra fue mayor que la búsqueda exacta y mantuvo promedios cercanos al conjunto completo.

12. Conclusiones

El proyecto HitSignal demostró que un pipeline de PySpark puede transformar datos musicales crudos en un sistema analítico para descubrimiento de oportunidades musicales. La limpieza inicial permitió conservar datos válidos y eliminar solamente registros problemáticos. El EDA identificó variables acústicas relevantes para construir un perfil de canciones populares.

La incorporación de `artists.csv` fue el punto metodológico central. Sin esta tabla, la detección de *hidden gems* quedaba limitada a la popularidad del track, lo cual generaba falsos positivos. Al usar popularidad y seguidores de artistas, fue posible filtrar canciones asociadas con artistas masivos y obtener una lista final más defendible.

La versión final de *hidden gems* produjo 11,613 canciones finales depuradas. Estas canciones son poco populares, relativamente recientes, de artistas no masivos y con perfil acústico alto. La búsqueda de similitud agregó interpretabilidad al asociar estas canciones con hits cercanos.

Finalmente, la demostración con LSH mostró cómo el enfoque puede escalar a espacios de comparación más grandes. En conjunto, HitSignal integra limpieza, análisis exploratorio, enriquecimiento, filtrado, scoring y búsqueda vectorial en un flujo coherente de datos masivos.

13. Bibliografía y recursos

- Apache Spark Documentation: <https://spark.apache.org/docs/latest/>
- PySpark SQL API Reference: <https://spark.apache.org/docs/latest/api/python/>
- Spark MLlib Feature Extraction and LSH: <https://spark.apache.org/docs/latest/ml-features.html>
- Spotify Tracks Dataset utilizado en el proyecto, consultado desde Kaggle.
- Notebooks desarrollados durante el proyecto: preparación, EDA, limpieza de artistas, enriquecimiento, hidden gems y similarity search.

A. Anexo: código esencial en PySpark

En este anexo se incluyen los fragmentos centrales del proyecto. Se omiten las celdas de inicialización de Spark y de carga de datos, como fue solicitado. Los fragmentos muestran la lógica principal de limpieza, variables derivadas, enriquecimiento, definición de *hidden gems* y búsqueda de similitud.

A.1. Limpieza de tracks y variables base

```

1 tracks_clean = (
2     tracks_raw
3     .filter(F.col("name").isNotNull())
4     .filter(F.col("tempo") > 0)
5     .filter(F.col("time_signature") > 0)
6 )
7
8 tracks_base = (
9     tracks_clean
10    .withColumn(
11        "release_year",
12        F.regexp_extract(F.col("release_date"), r"^\d{4}", 1).cast("int")
13    )
14    .withColumn(
15        "release_decade",
16        (F.floor(F.col("release_year") / 10) * 10).cast("int")
17    )
18    .withColumn("duration_min", F.round(F.col("duration_ms") / 60000, 2))
19    .withColumn("is_hit", F.when(F.col("popularity") >= 70, 1).otherwise(0))
20    .withColumn(
21        "popularity_class",
22        F.when(F.col("popularity") < 40, "baja")
23        .when(F.col("popularity") < 70, "media")
24        .otherwise("alta")
25    )
26 )

```

Listing 1: Limpieza conservadora y variables derivadas

A.2. Puntuación acústica

```

1 W = {
2     "acousticness": 0.25988213907708085,
3     "loudness": 0.23016439975443223,
4     "energy": 0.21218715648715247,

```

```

5     "instrumentalness": 0.16620344705647533,
6     "danceability": 0.13156285762485917
7 }
8
9 tracks_enriched = (
10     tracks_enriched
11     .withColumn(
12         "hit_score_weighted",
13         F.round(
14             (
15                 (1 - F.col("acousticness_norm")) * F.lit(W["acousticness"
16 ]) +
17                 F.col("loudness_norm") * F.lit(W["loudness"]) +
18                 F.col("energy_norm") * F.lit(W["energy"]) +
19                 (1 - F.col("instrumentalness_norm")) * F.lit(W["
instrumentalness"]) +
20                 F.col("danceability_norm") * F.lit(W["danceability"])
21             ) * 100,
22             2
23         )
24     )

```

Listing 2: Cálculo de score acústico ponderado

A.3. Join entre tracks y artistas

```

1 tracks_artists_exploded = (
2     tracks_base
3     .withColumn("artist_id", F.explode("artist_ids")))
4 )
5
6 tracks_artist_joined = (
7     tracks_artists_exploded
8     .join(artists_ref, on="artist_id", how="left")
9 )
10
11 artist_features_by_track = (
12     tracks_artist_joined
13     .groupBy("track_id")
14     .agg(
15         F.collect_set("artist_id").alias("artist_ids"),
16         F.collect_set("artist_name").alias("artist_names_ref"),
17         F.max("artist_popularity").alias("max_artist_popularity"),
18         F.max("artist_followers").alias("max_artist_followers"),
19         F.collect_set("artist_popularity_class").alias("
artist_popularity_classes"),
20         F.collect_set("artist_size_class").alias("artist_size_classes")

```

```

21 )
22 )

```

Listing 3: Explode de artistas y agregación por track

A.4. Definición final de Hidden Gems V3

```

1  TRACK_POPULARITY_MAX = 40
2  MIN_RELEASE_YEAR = 2010
3  MIN_HIT_SCORE = 80
4  MAX_ARTIST_POPULARITY = 60
5  MAX_ARTIST_FOLLOWERS = 500000
6
7  tracks_hg_flags = (
8      tracks_enriched
9      .withColumn("is_low_track_popularity", F.col("popularity") <
10         TRACK_POPULARITY_MAX)
11      .withColumn("is_recent_enough", F.col("release_year") >=
12         MIN_RELEASE_YEAR)
13      .withColumn("is_high_hit_score", F.col("hit_score_weighted") >=
14         MIN_HIT_SCORE)
15      .withColumn(
16          "has_artist_match",
17          F.col("max_artist_popularity").isNull() &
18          F.col("max_artist_followers").isNull()
19      )
20      .withColumn("is_artist_too_popular", F.col("max_artist_popularity")
21         >= MAX_ARTIST_POPULARITY)
22      .withColumn("is_artist_too_massive", F.col("max_artist_followers") >=
23         MAX_ARTIST_FOLLOWERS)
24 )
25
26 hidden_gems_v3_candidates = (
27     tracks_hg_flags
28     .filter(
29         F.col("is_low_track_popularity") &
30         F.col("is_recent_enough") &
31         F.col("is_high_hit_score") &
32         F.col("has_artist_match") &
33         (~F.col("is_artist_too_popular")) &
34         (~F.col("is_artist_too_massive"))
35     )
36 )

```

Listing 4: Filtros finales de Hidden Gems V3

A.5. Recencia, score final y deduplicación

```

1 lambda_decay = math.log(2) / 10
2
3 hidden_gems_v3_candidates = (
4     hidden_gems_v3_candidates
5     .withColumn("song_age", F.lit(max_year) - F.col("release_year"))
6     .withColumn("recency_score", F.round(F.exp(-F.lit(lambda_decay) * F.
7         col("song_age")) * 100, 2))
8     .withColumn(
9         "hidden_gem_score",
10        F.round(0.80 * F.col("hit_score_weighted") + 0.20 * F.col("
11            recency_score"), 2)
12    )
13    .withColumn(
14        "track_name_norm",
15        F.lower(F.trim(F.regexp_replace(F.col("track_name"), r"[\p{L}\p{
16            N}]+", " ")))
17    )
18    .withColumn("artist_key", F.concat_ws("_", F.array_sort(F.col("
19        artist_ids"))))
20 )
21
22 w_dedup = Window.partitionBy("track_name_norm", "artist_key") \
23     .orderBy(F.desc("hidden_gem_score"), F.desc("hit_score_weighted"))
24
25 hidden_gems_v3 = (
26     hidden_gems_v3_candidates
27     .withColumn("rn", F.row_number().over(w_dedup))
28     .filter(F.col("rn") == 1)
29     .drop("rn")
30 )

```

Listing 5: Recency score, hidden gem score y deduplicación

A.6. Búsqueda exacta de similitud

```

1 distance_expr = None
2
3 for c in weighted_feature_cols:
4     diff_sq = F.pow(F.col(f"gem_{c}") - F.col(f"hit_{c}"), 2)
5     distance_expr = diff_sq if distance_expr is None else distance_expr +
6         diff_sq
7
8 similarity_exact_sample = (
9     gems_sample_renamed
10    .crossJoin(hits_sample_renamed)
11    .filter(F.col("gem_artists") != F.col("hit_artists"))

```

```

11     .withColumn("distance", F.sqrt(distance_expr))
12 )
13
14 nearest_hit_window = Window.partitionBy("gem_track_id").orderBy(F.asc("
    distance"))
15
16 hidden_gem_nearest_hit_exact_sample = (
17     similarity_exact_sample
18     .withColumn("similarity_rank", F.row_number().over(nearest_hit_window
19 ))
20     .filter(F.col("similarity_rank") == 1)
21 )

```

Listing 6: Distancia euclidiana ponderada entre hidden gems y hits

A.7. Búsqueda aproximada con LSH

```

1 brp_lsh = BucketedRandomProjectionLSH(
2     inputCol="features",
3     outputCol="hashes",
4     bucketLength=0.25,
5     numHashTables=1
6 )
7
8 brp_lsh_model = brp_lsh.fit(hits_lsh_sample)
9
10 hits_lsh_hashed = brp_lsh_model.transform(hits_lsh_sample)
11 gems_lsh_hashed = brp_lsh_model.transform(gems_lsh_sample)
12
13 lsh_candidates_raw = brp_lsh_model.approxSimilarityJoin(
14     gems_lsh_hashed,
15     hits_lsh_hashed,
16     0.15,
17     distCol="distance"
18 )
19
20 nearest_lsh_window = Window.partitionBy("gem_track_id").orderBy(F.asc("
    distance"))
21
22 hidden_gem_nearest_hit_lsh = (
23     lsh_candidates
24     .withColumn("similarity_rank", F.row_number().over(nearest_lsh_window
25 ))
26     .filter(F.col("similarity_rank") == 1)
27 )

```

Listing 7: BucketedRandomProjectionLSH sobre muestra estratificada